

Informe TP N°7

- **Objetivo del TP:** Obtener los conocimientos necesarios para poder configurar un socket en Python para crear una conexión Cliente/Servidor
- **Materiales:** Compilador Python y el comando Netstat de Windows.
- **Escenario:** Generar 2 programas en Python, uno de un servidor y otro de un cliente. Realizar la conexión entre los dos y luego comprobar las conexiones abiertas mediante el comando netstat.
- **Actividad:**
 1. Crear un programa de conexión de socket (servidor)

```
1  import socket
2  s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
3  s.bind(("127.0.0.1", 7777))
4  s.listen(1)
5
6  print ("Servidor de Chat\n")
7
8  while True:
9      print ("Esperando conexión del cliente...")
10     sc, addr = s.accept()
11     print ("Cliente conectado desde: ", addr)
12
13     while True:
14         recibido = sc.recv(1024)
15         if recibido == "quit":
16             break
17         print ("Recibido: ", recibido)
18
19         nuestra_respuesta = "Hola cliente, yo soy el servidor"
20         sc.send(nuestra_respuesta.encode('utf-8'))
21
22     print ("Adios")
23     sc.close()
24     s.close()
```

2. Crear un programa de conexión de socket (cliente).

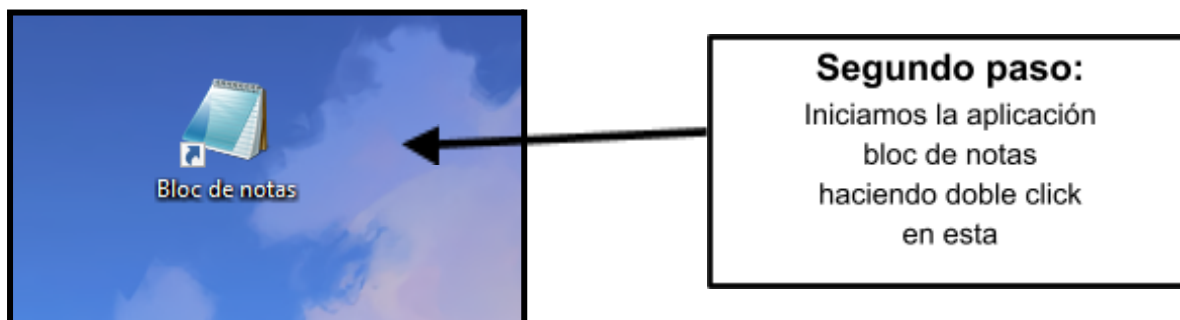
```
1  import socket
2
3  socket_cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4  socket_cliente.connect(("127.0.0.1", 7777))
5
6  while True:
7      mensaje = str(input(">> "))
8      socket_cliente.send(mensaje.encode('utf-8'))
9
10     recibido = socket_cliente.recv(1024)
11     print("Recibido: ", recibido)
12
13     print ("Adios")
14     socket_cliente.close()
```

3. Ejecutar el siguiente comando cmd -> “netstat -ano > C:\temp\sinconexion.txt”
4. Ejecutar primero el programa de servidor y luego el programa de cliente.
5. Verificar que la conexión se haya establecido de forma correcta.
6. Ejecutar el siguiente comando cmd -> “netstat -ano > C:\temp\conconexion.txt”
7. Comparar los TXT sinconexion y conconexion. ¿Qué diferencias encontramos?
8. Desarrollar una conclusión del trabajo realizado.

ACTIVIDAD:

1) Crear un programa de conexión socket (servidor)

Primer paso: Descargamos Python desde el siguiente enlace: [Python](#)



Tercer paso:

Dentro de la aplicación bloc de notas escribimos el siguiente código de conexión de socket (Servidor)

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 7777))
s.listen(1)

print("Servidor de Chat\n")

while True:
    print("Esperando conexión del cliente...")
    sc, addr = s.accept()
    print("Cliente conectado desde: ", addr)

    while True:
        recibido = sc.recv(1024)
        if recibido == "quit":
            break
        print("Recibido: ", recibido)

    nuestra_respuesta = "Hola cliente, yo soy el servidor"
```

```
sc.send(nuestra_respuesta.encode('utf-8'))
```

```
print("Adios")
sc.close()
s.close()
```

Nos debería quedar algo así:

```
*Sin título: Bloc de notas
Archivo Edición Formato Ver Ayuda
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 7777))
s.listen(1)

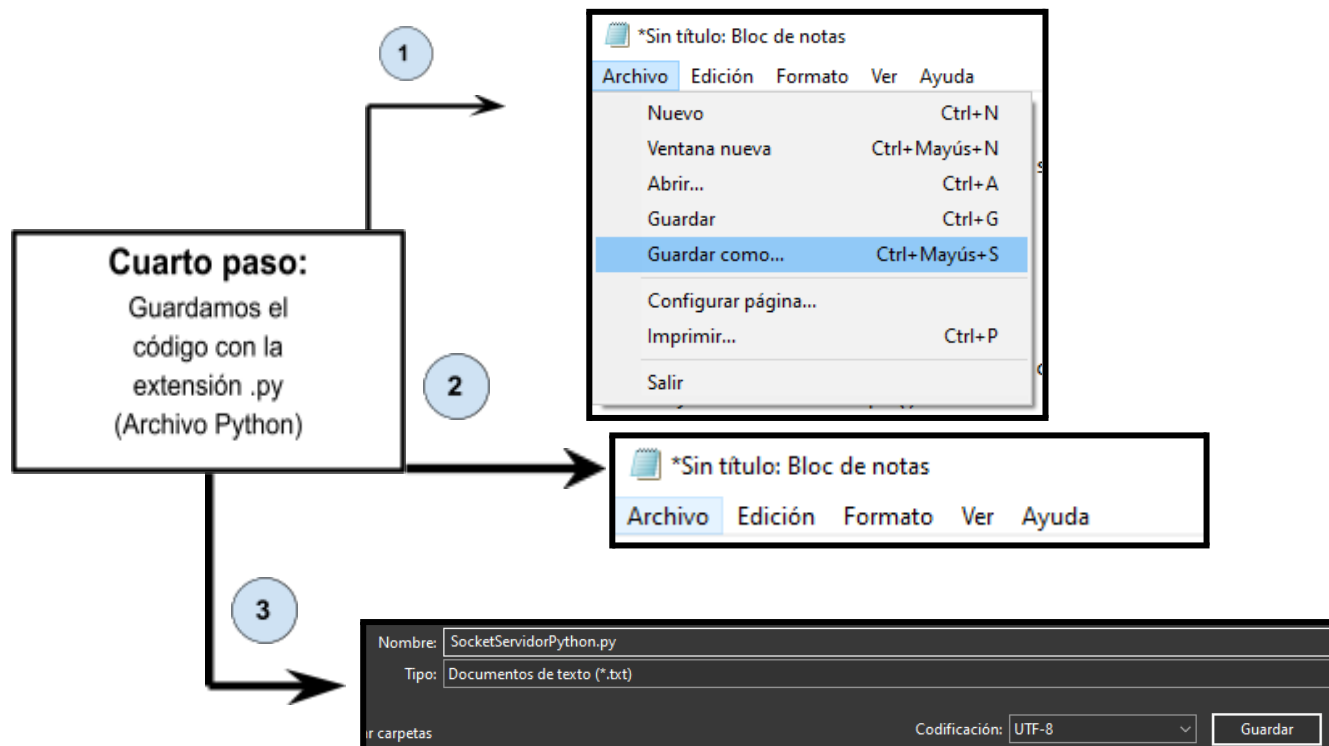
print("Servidor de Chat\n")

while True:
    print("Esperando conexión del cliente...")
    sc, addr = s.accept()
    print("Cliente conectado desde: ", addr)

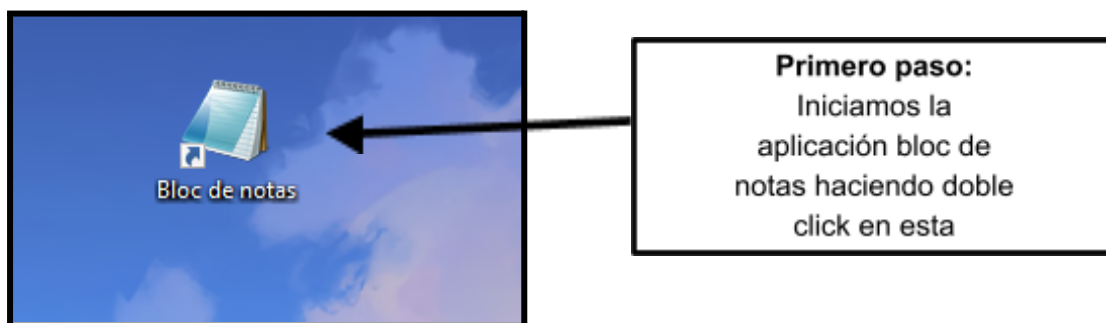
    while True:
        recibido = sc.recv(1024)
        if recibido == "quit":
            break
        print("Recibido: ", recibido)

        nuestra_respuesta = "Hola cliente, yo soy el servidor"
        sc.send(nuestra_respuesta.encode('utf-8'))

print("Adios")
sc.close()
s.close()
```



2) Crear un programa de conexión socket (cliente)



Segundo paso: Dentro de la aplicación bloc de notas escribimos el siguiente código de conexión de socket (cliente)

```
import socket
```

```
socket_cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
socket_cliente.connect(("127.0.0.1", 7777))
```

```
while True:
```

```
    mensaje = str(input(">> "))
    socket_cliente.send(mensaje.encode('utf-8'))
```

```
    recibido = socket_cliente.recv(1024)
    print("Recibido: ", recibido)
```

```
print("Adios")
socket_cliente.close()
```

Nos debería quedar algo así:

```
*Sin título: Bloc de notas
Archivo Edición Formato Ver Ayuda

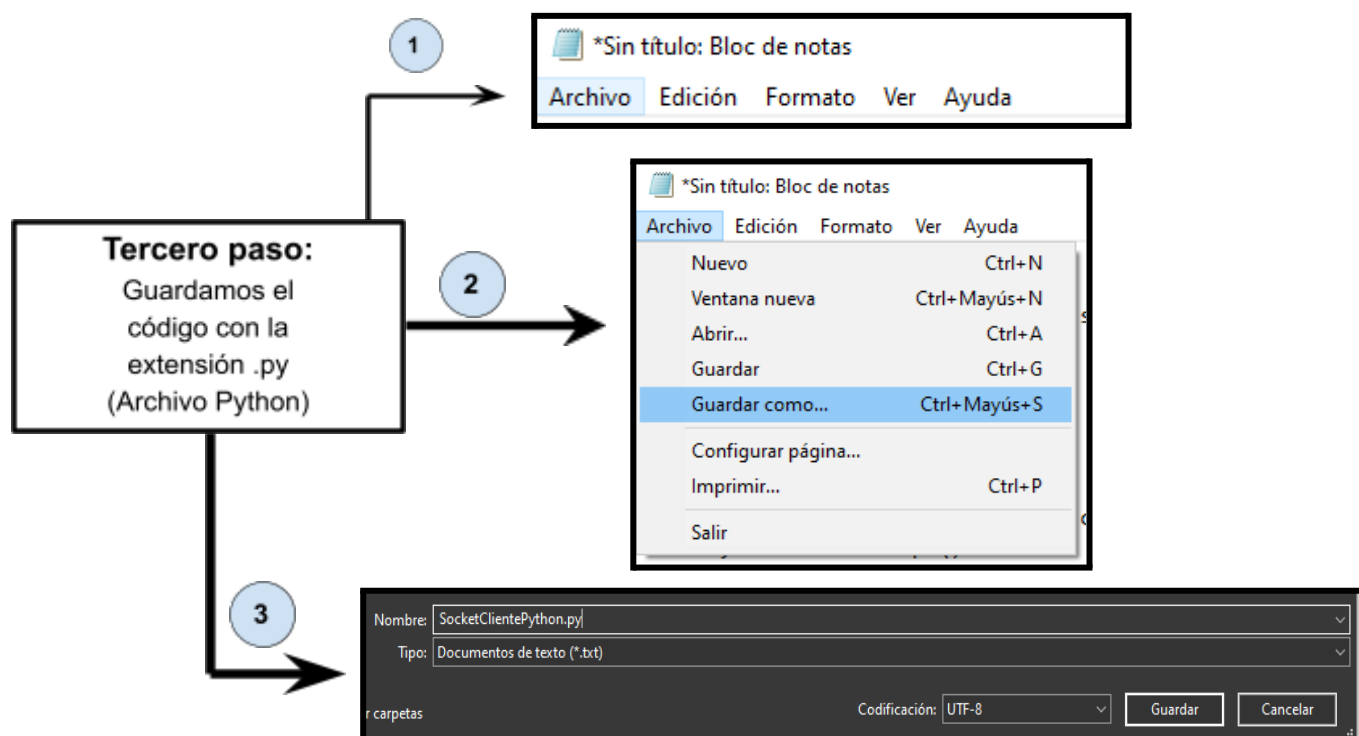
import socket

socket_cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
socket_cliente.connect(("127.0.0.1", 7777))

while True:
    mensaje = str(input(">> "))
    socket_cliente.send(mensaje.encode('utf-8'))

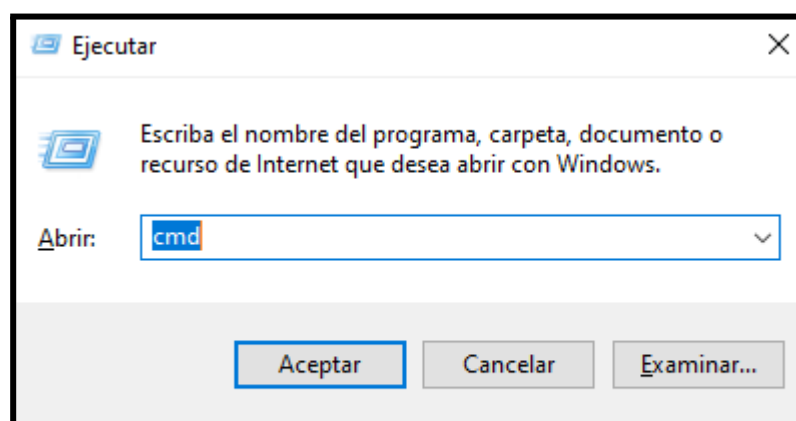
    recibido = socket_cliente.recv(1024)
    print("Recibido: ", recibido)

print("Adios")
socket_cliente.close()
```



3) Ejecutar el siguiente comando cmd > “netstat -ano > C:\temp\sinconexion.txt”

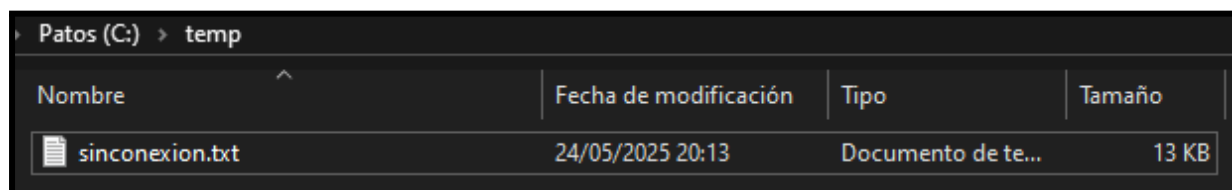
Primer paso: Nos dirigimos a la ventana de ejecutar con la combinación de teclas “Windows+R”, escribimos “cmd” y le damos a aceptar para iniciarlo.



Segundo paso: Dentro del “cmd” escribimos el siguiente comando y apretamos “Enter”

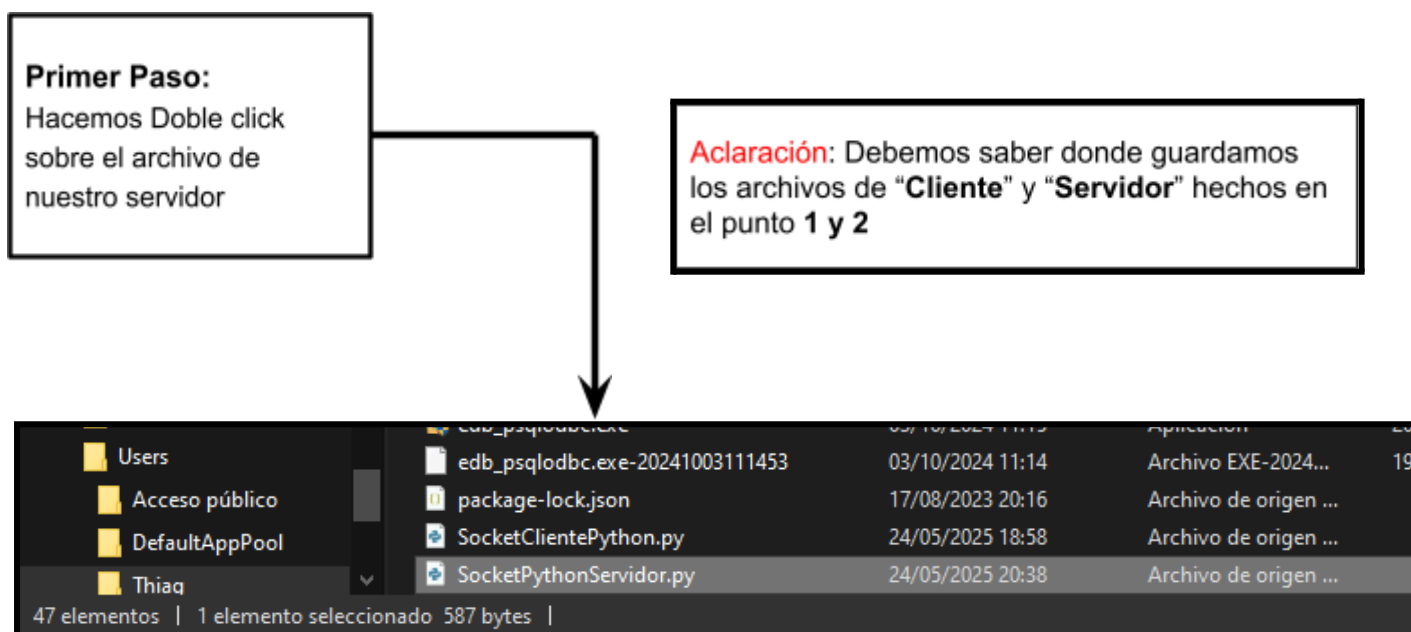
```
C:\Users\Thiag>netstat -ano > C:\temp\sinconexion.txt
```

Esto que hicimos nos va a crear un archivo .txt llamado "sinconexion" en el cual vamos a enlazar y guardar las diferentes conexiones activas, sin estar activo el socket servidor-cliente que configuramos anteriormente.

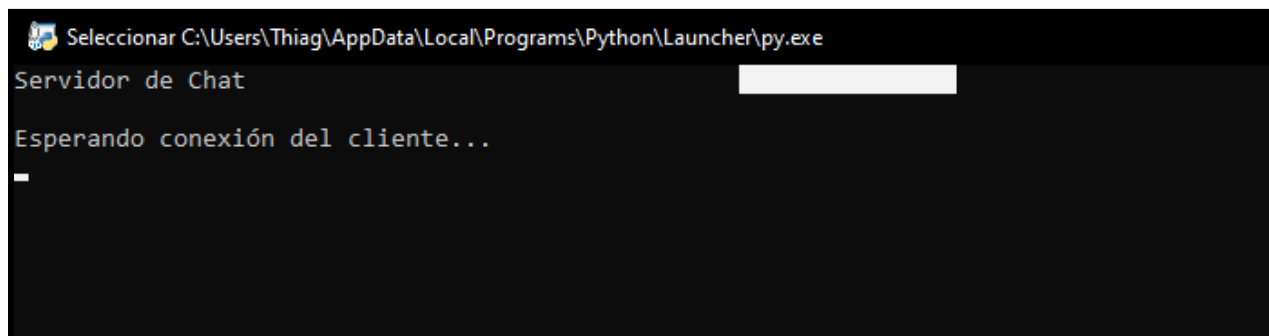


Nombre	Fecha de modificación	Tipo	Tamaño
sinconexion.txt	24/05/2025 20:13	Documento de te...	13 KB

4) Ejecutar primero el programa de servidor y luego el programa de cliente.



Nos va a abrir una pantalla como esta:



```

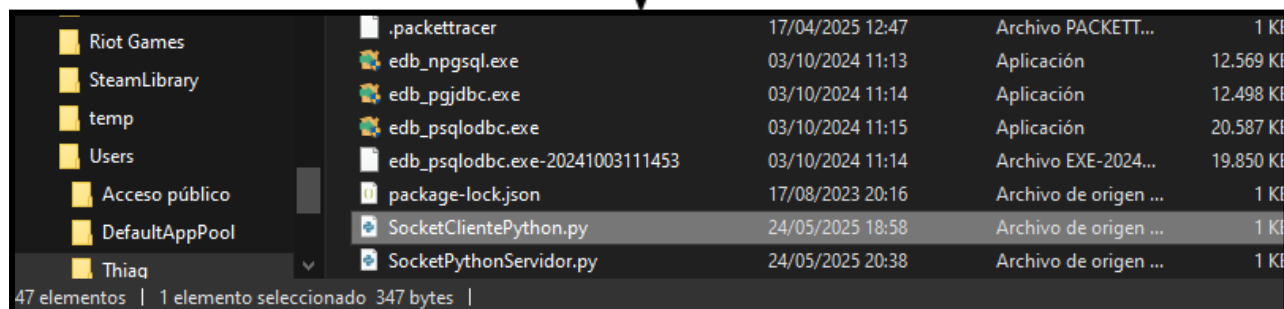
Selecc... C:\Users\Thiaq\AppData\Local\Programs\Python\Launcher\py.exe
Servidor de Chat
Esperando conexión del cliente...

```


Segundo Paso:

Hacemos Doble click sobre el archivo de nuestro cliente

Aclaración: Debemos saber donde guardamos los archivos de "Cliente" y "Servidor" hechos en el punto 1 y 2



Nombre	Fecha	Tamaño	Detalles
.packettracer	17/04/2025 12:47	1 KI	Archivo PACKETT...
edb_npgsql.exe	03/10/2024 11:13	12.569 KI	Aplicación
edb_pgjdbc.exe	03/10/2024 11:14	12.498 KI	Aplicación
edb_psqldb.exe	03/10/2024 11:15	20.587 KI	Aplicación
edb_psqldb.exe-20241003111453	03/10/2024 11:14	19.850 KI	Archivo EXE-2024...
package-lock.json	17/08/2023 20:16	1 KI	Archivo de origen ...
SocketClientePython.py	24/05/2025 18:58	1 KI	Archivo de origen ...
SocketPythonServidor.py	24/05/2025 20:38	1 KI	Archivo de origen ...

47 elementos | 1 elemento seleccionado 347 bytes |

5) Verificar que la conexión se haya establecido de forma correcta.

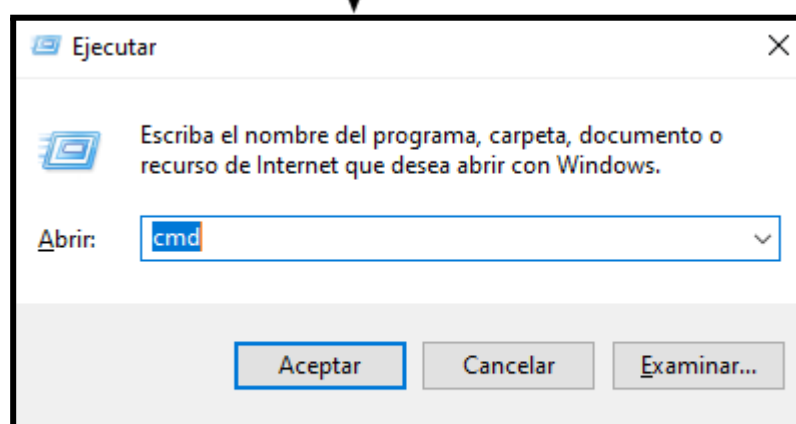
Para verificarlo veremos en la pantalla del servidor el mensaje "Cliente conectado desde ("Número de IP, puerto abierto")"

```
C:\Users\Thiaa\AppData\Local\Programs\Python\Launcher\py.exe
Servidor de Chat

Esperando conexión del cliente...
Cliente conectado desde: ('127.0.0.1', 39808)
_
```

6) Ejecutar el siguiente comando cmd > “netstat -ano > C:\temp\conconexion.txt”

Primer paso: Nos dirigimos a la ventana de ejecutar con la combinación de teclas “Windows+R”, escribimos “cmd” y le damos a aceptar para iniciarlo.



Segundo paso: Dentro del “cmd” escribimos el siguiente comando y apretamos “Enter”

```
C:\Users\Thiag>netstat -ano > C:\temp\conconexion.txt
```

Esto que hicimos nos va a crear un archivo .txt llamado “conconexion” en el cual vamos a enlistar y guardar las diferentes conexiones activas, luego de haber establecido la conexión cliente - servidor en el socket que creamos.

> Este equipo > Patos (C:) > temp			
programa	Nombre	Fecha de modificación	Tipo
programa (x)	conconexion.txt	24/05/2025 21:11	Documento de te...
	sinconexion.txt	24/05/2025 20:13	Documento de te...

7) Comparar los TXT sinconexion y conconexion. ¿Qué diferencias encontramos?

Al comparar los archivos .txt “sinconexion” y “conconexion” vemos que se creó un nuevo socket y se estableció una nueva conexión con los siguientes datos.

TCP	127.0.0.1:7777	0.0.0.0:0	LISTENING	20284
TCP	127.0.0.1:7777	127.0.0.1:39808	ESTABLISHED	20284
TCP	127.0.0.1:39808	127.0.0.1:7777	ESTABLISHED	3128

Aca podemos ver como el servidor empezó a buscar clientes que se conecten a él, en el renglón que dice “listening”. En el segundo renglón podemos ver que puerto se abrió para el socket, y en el tercer renglón como se establece la conexión del cliente con el servidor.

8) CONCLUSIÓN

Pusimos en práctica los conceptos fundamentales de la programación de sockets en Python, así como aprender a verificar conexiones mediante la herramienta **NETSTAT**. A través de las distintas etapas, logramos no solo entender cómo se establecen y gestionan las conexiones entre un servidor y un cliente, sino también analizar el tráfico de red de manera detallada.

Al comparar los archivos generados con el comando **NETSTAT** antes y después de ejecutar los programas, observamos diferencias claras. Después de ejecutar el servidor cliente, se estableció una conexión activa (ESTABLISHED) entre ambos programas, con identificadores de proceso (PID) asociados a cada conexión. Esto confirmó que los programas de cliente y servidor funcionaban correctamente y se comunicaban tal como esperábamos.

Esta experiencia nos permitió ganar conocimientos clave, como el uso de puertos para establecer comunicación, la importancia de utilizar herramientas de monitoreo como **NETSTAT** para verificar el correcto funcionamiento de un sistema, y cómo identificar y solucionar posibles problemas de red. Además, la práctica de comparar los estados "sin conexión" y "con conexión" nos dejó observar los cambios que hay en la red.